

Lab 11:

Enterprise Disaster Recovery and Rollback Lab on Astronomer Astro + Airflow

Company: FinSure Analytics

Platform:

- Astronomer Astro
- Apache Airflow
- PostgreSQL metadata database
- Astro CLI
- AWS EC2 Ubuntu VM

Production Problem:

A new deployment was pushed containing:

- DAG syntax issue
- Incorrect task dependency
- Corrupted Variables entry
- Deployment image mismatch

Impact:

- DAGs failing continuously
- Scheduler instability
- Historical runs stuck
- Business reports delayed

Engineering Team Goals:

1. Backup metadata database
2. Restore environment if failure occurs
3. Rollback deployment safely
4. Recover DAG execution states
5. Validate DR workflow
6. Document incident response
- 7.

Step 1 — Create Project Structure

Create Astro Project

```
mkdir astro-dr-lab  
cd astro-dr-lab
```

Initialize project:

```
astro dev init
```

Expected structure:

```
astro-dr-lab/  
├── dags/  
├── plugins/  
├── include/  
├── Dockerfile  
├── requirements.txt  
└── airflow_settings.yaml
```

Step 2 — Create Production DAG

Create DAG:

```
cd dags  
vim financial_etl_pipeline.py
```

Add:

```
from airflow import DAG  
from airflow.operators.python import PythonOperator  
from datetime import datetime  
import time
```

```
default_args = {  
    "owner": "finance-team"  
}
```

```
def extract():  
    print("Extracting transaction data")  
    time.sleep(5)
```

```
def transform():  
    print("Transforming financial records")  
    time.sleep(5)
```

```

def load():
    print("Loading into warehouse")
    time.sleep(5)

with DAG(
    dag_id="financial_etl_pipeline",
    start_date=datetime(2025, 1, 1),
    schedule="@hourly",
    catchup=False,
    default_args=default_args
) as dag:

    extract_task = PythonOperator(
        task_id="extract",
        python_callable=extract
    )

    transform_task = PythonOperator(
        task_id="transform",
        python_callable=transform
    )

    load_task = PythonOperator(
        task_id="load",
        python_callable=load
    )

    extract_task >> transform_task >> load_task

```

Step 3 — Start Local Astro Environment

astro dev start

Verify containers:

docker ps

Expected:

- scheduler
- webserver
- postgres
- triggerer

Open UI:

http://localhost:8080

Step 4 — Trigger DAG

Trigger manually:

```
astro dev run dags trigger financial_etl_pipeline
```

Check runs:

```
astro dev run dags list-runs -d financial_etl_pipeline
```

Step 5 — Backup Metadata Database

This is a critical DR operation.

Understand What Metadata Contains

Airflow metadata DB stores:

- DAG run history
- Task instance states
- Variables
- Connections
- XComs
- Users
- Logs metadata

Losing metadata means:

- No historical tracking
- Lost retries
- Scheduler inconsistency
- Broken orchestration

Step 6 — Identify Postgres Container

List containers:

```
docker ps
```

Find:

astro_dr_lab_postgres_1

Step 7 — Backup PostgreSQL Metadata

Create backup directory:

```
mkdir ~/airflow-backups
```

Run backup:

```
docker exec astro_dr_lab_postgres_1 pg_dump -U postgres postgres > ~/airflow-backups/metadata_backup.sql
```

Verify:

```
ls ~/airflow-backups
```

Expected:

```
metadata_backup.sql
```

Step 8 — Simulate Disaster Scenario

Now simulate a broken deployment.

Edit DAG:

```
vim dags/financial_etl_pipeline.py
```

Introduce intentional syntax error:

```
def transform(
```

Save file.

Step 9 — Deploy Broken DAG

Restart environment:

```
astro dev restart
```

Observe:

- Scheduler import errors
- DAG disappears

- Deployment unstable

Check logs:

```
astro dev logs scheduler
```

Expected:

- Python syntax errors
- DAG parsing failures

Step 10 — Simulate Corrupted Metadata

Access PostgreSQL shell:

```
docker exec -it astro_dr_lab_postgres_1 psql -U postgres
```

Inside PostgreSQL:

```
UPDATE dag SET is_paused = true;
```

Exit:

```
\q
```

Now DAG behavior becomes inconsistent.

Step 11 — Restore Metadata Database

This simulates enterprise DR restoration.

Stop Astro Environment

```
astro dev stop
```

Remove Existing Metadata

```
docker volume ls
```

Locate postgres volume.

Remove:

```
docker volume rm astro_dr_lab_postgres-db-volume
```

Restart Environment

`astro dev start`

New clean DB initializes.

Restore Backup

Copy backup into container:

```
docker cp ~/airflow-backups/metadata_backup.sql astro_dr_lab_postgres_1:/tmp/
```

Restore:

```
docker exec -it astro_dr_lab_postgres_1 psql -U postgres -d postgres -f  
/tmp/metadata_backup.sql
```

Step 12 — Validate Recovery

Check DAGs:

```
astro dev run dags list
```

Check DAG runs:

```
astro dev run dags list-runs -d financial_etl_pipeline
```

Verify:

- Historical runs restored
 - DAG state recovered
 - Metadata available
-

Module 2 — Rollback and Recovery

Now simulate production rollback.

Step 13 — DAG Rollback Strategy

Enterprise rollback strategies:

Strategy	Description
Git Rollback	Revert commit
DAG Versioning	Restore previous DAG
Container Rollback	Restore previous image
Metadata Recovery	Restore DB snapshot
Selective Task Recovery	Clear failed tasks only

Step 14 — Implement DAG Versioning

Create backup DAG version:

```
cp dags/financial_etl_pipeline.py dags/financial_etl_pipeline_v1.py
```

Create broken v2:

```
cp dags/financial_etl_pipeline.py dags/financial_etl_pipeline_v2.py
```

Modify v2 with bad dependency:

```
load_task >> extract_task
```

This creates cyclic dependency.

Step 15 — Observe Failure

Restart:

```
astro dev restart
```


Check logs:

```
astro dev logs scheduler
```

Expected:

- Cycle detected
 - DAG import failure
-

Step 16 — Rollback Failed Deployment

Delete faulty DAG:

```
rm dags/financial_etl_pipeline_v2.py
```

Restore stable version:

```
cp dags/financial_etl_pipeline_v1.py dags/financial_etl_pipeline.py
```

Restart:

```
astro dev restart
```

Verify:

- DAG loads successfully
 - Scheduler healthy
-

Step 17 — Deployment Rollback Strategy

Real enterprises rollback:

- Docker image tags
- Helm revisions
- Git tags
- CI/CD artifacts

Simulate image rollback.

Step 18 — Create Git Commit History

Initialize Git:

```
git init
git add .
git commit -m "Stable production deployment"
```

Introduce bad deployment:

```
echo "broken deployment" >> requirements.txt
```

Commit:

```
git add .
git commit -m "Faulty deployment"
```

Step 19 — Rollback Deployment

View commits:

```
git log --oneline
```

Rollback:

```
git reset --hard HEAD~1
```

This restores stable deployment.

Step 20 — Failure Recovery Workflow

Enterprise recovery workflow:

Failure Detected



Incident Opened



Logs Collected



Root Cause Identified



Rollback Initiated

↓
Metadata Restored
↓
DAG Recovered
↓
Validation Testing
↓
Incident Closed

Step 21 — Recover DAG Execution

Simulate task failure.

Modify transform function:

```
def transform():  
    raise Exception("Transformation failed")
```

Restart:

```
astro dev restart
```

Trigger DAG:

```
astro dev run dags trigger financial_etl_pipeline
```

Check failed tasks:

```
astro dev run tasks states-for-dag-run financial_etl_pipeline <run_id>
```

Step 22 — Recover Failed DAG Execution

Fix DAG code:

```
def transform():  
    print("Transformation fixed")
```

Restart:

```
astro dev restart
```

Clear failed task:

```
astro dev run tasks clear financial_etl_pipeline -t transform
```

Re-run DAG.

Expected:

- Failed tasks resume
 - DAG completes successfully
-

Step 23 — Incident Response Playbook

Create incident playbook:

1. Detect scheduler/webserver issue
 2. Check deployment logs
 3. Identify affected DAGs
 4. Pause unstable DAGs
 5. Backup metadata immediately
 6. Rollback deployment
 7. Restore metadata if required
 8. Clear failed tasks
 9. Validate scheduler health
 10. Resume operations
 11. Document RCA
-

Step 24 — DR Workflow Validation

Participants must validate:

Validation	Expected
Metadata restored	DAG history available
DAG rollback works	Scheduler healthy
Deployment rollback works	Stable environment
Failed DAG recovered	DAG succeeds

Incident handled

Logs + RCA available